

Partie 08 — Mesurer et améliorer les performances

INF-STAR — Programmation statistique avec R

Oskar Laverny

October 5, 2026

Aix-Marseille Université

Objectifs

Vous saurez :

- expliquer pourquoi on mesure avant d'optimiser ;
- utiliser `system.time()` pour un ordre de grandeur ;
- comparer plusieurs expressions avec `bench::mark()` ;
- interpréter temps médian, itérations par seconde et mémoire allouée ;
- appliquer quelques améliorations simples et sûres.

Le profilage avec `profvis` est un complément facultatif. Le socle pratique est la comparaison de trois versions d'un même calcul.

Pourquoi mesurer ?



Pour un script qui s'exécute en 0,2 seconde, gagner quelques microsecondes n'a souvent aucun intérêt.

Priorités :

1. résultat correct ;
2. code lisible et testable ;
3. mesure d'un vrai problème de performance ;
4. optimisation ciblée.

Ne pas optimiser à l'intuition

Le code qui paraît « compliqué » n'est pas forcément le goulot d'étranglement.

Mesurer permet de répondre à deux questions différentes :

- **combien de temps** prend une opération ?
- **où** le programme passe-t-il son temps ?

Benchmarking et profiling ne sont donc pas la même chose.

Première mesure



```
system.time()
```

```
x <- runif(1e6)

system.time({
  y <- sqrt(x)
})
```

`system.time()` est utile pour chronométrer une opération suffisamment longue.

Il est moins adapté aux très petites expressions.

`Sys.time()` pour un bloc

```
debut <- Sys.time()  
# ... traitement ...  
fin <- Sys.time()  
  
fin - debut
```

Simple et pratique pour un temps « mur » global, moins précis pour comparer des opérations très rapides.

bench



Microbenchmark moderne

```
install.packages("bench")  
library(bench)
```

Puis :

```
x <- runif(1000)  
  
bench::mark(  
  sqrt(x),  
  x^0.5  
)
```

`bench::mark()` répète les expressions et résume leur comportement.

Colonnes utiles :

- `min` : meilleur temps observé ;
- `median` : temps typique ;
- `itr/sec` : itérations par seconde ;
- `mem_alloc` : mémoire allouée ;
- `n_gc / gc/sec` : activité du ramasse-miettes.

Pour un cours introductif, concentrez-vous surtout sur **median**, **itr/sec** et **mem_alloc**.

Comparer des résultats équivalents

```
bench::mark(  
  mean(x),  
  sum(x) / length(x)  
)
```

Par défaut, `bench::mark()` vérifie que les expressions retournent des résultats compatibles.

Si vous comparez volontairement des sorties différentes :

```
bench::mark(expr1, expr2, check = FALSE)
```

Une différence de 500 ns peut sembler énorme en ratio et être insignifiante dans une analyse qui s'exécute une fois.

Question utile : **combien de fois cette opération est-elle réellement appelée ?**

Optimiser 1 ms répété un million de fois peut compter. Optimiser 1 μ s appelé dix fois, beaucoup moins.

Profiling



Benchmark : « quelle version est plus rapide ? »

Profiling : « quelle partie de mon programme est lente ? »

Le package actuel recommandé dans l'écosystème R est `profvis` :

```
install.packages("profvis")  
library(profvis)
```

```
profvis::profvis({  
  # appel d'une fonction ou d'un traitement réaliste  
  resultat <- mon_calcul(donnees)  
})
```

profvis fournit une vue interactive du temps passé dans les appels de fonctions et les lignes de code.

Profilons un **cas réaliste**, pas un micro-exemple artificiel.

Améliorations simples

Préallocation

Version qui fait grandir l'objet :

```
res <- numeric(0)
for (i in 1:10000) {
  res <- c(res, i^2)
}
```

Version préallouée :

```
res <- numeric(10000)
for (i in seq_along(res)) {
  res[i] <- i^2
}
```

Mesurez la différence au lieu de la supposer.

```
res <- (1:10000)^2
```

Une opération vectorisée peut être à la fois plus concise et plus rapide car elle s'appuie sur des implémentations optimisées.

Mais une boucle claire est préférable à une vectorisation incompréhensible.

Comparez :

```
apply(m, 2, mean)  
colMeans(m)
```

Une fonction spécialisée exprime mieux l'intention et peut éviter le coût d'une fonction générale.

Ne pas recalculer inutilement

À éviter :

```
for (i in seq_along(x)) {  
  seuil <- mean(x)  
  # ...  
}
```

Si `mean(x)` ne dépend pas de `i`, calculez-la avant la boucle.

Mémoire



`bench::mark()` indique `mem_alloc`.

Cela suffit souvent à repérer une solution qui crée beaucoup plus d'objets temporaires qu'une autre.

Pour ce cours, on ne cherche pas à maîtriser toute la gestion mémoire interne de R : on apprend à **observer** puis à corriger un problème concret.

Une démarche reproductible



1. écrire une version correcte ;
2. créer un cas représentatif ;
3. mesurer ;
4. identifier le goulot ;
5. modifier une chose à la fois ;
6. vérifier que le résultat n'a pas changé ;
7. re-mesurer.

Une optimisation non mesurée est une hypothèse.

Outils à retenir



```
system.time(...)      # chronométrage simple  
bench::mark(...)      # comparaison précise de petites expressions  
profvis::profvis(...) # localiser les goulots dans un calcul plus large
```

Certaines autres interface plus vieilles comme `microbenchmark` ou `lineprof` sont maintenant obsoletes et remplacées par `bench` et `profvis`.

Atelier — comparer trois écritures

1. Construire les carrés de `1:n` avec une boucle qui agrandit le résultat.
2. Préallouer le vecteur résultat et refaire la boucle.
3. Écrire l'expression vectorisée.
4. Vérifier les résultats avec `all.equal()`.
5. Mesurer les trois versions sur les mêmes entrées.

Commencer avec `n <- 5000`. Pour aller plus loin, changer la taille et expliquer pourquoi le classement ou l'ampleur des différences peut varier.

- Advanced R — Measuring performance : <https://adv-r.hadley.nz/perf-measure.html>
- Advanced R — Improving performance : <https://adv-r.hadley.nz/perf-improve.html>
- Package bench : <https://bench.r-lib.org/>
- Package profvis : <https://rstudio.github.io/profvis/>